

Relatório Técnico – Unidade 2

Disciplina: Introdução a Técnicas de Programação (ITP) – 2025.2

Projeto: Sistema de Gerenciamento de Biblioteca em C

Aluno: Elton Gustavo da Silva Freitas

Matrícula: 20250024590

1. Introdução

Nesta unidade, o objetivo foi aprimorar o Sistema de Gerenciamento de Biblioteca desenvolvido anteriormente, aplicando os conceitos de estruturas de dados e manipulação de memória. O programa simula o controle de uma pequena biblioteca, permitindo cadastrar, listar, buscar e alugar livros de forma dinâmica e organizada.

O foco esteve no uso de *structs*, alocação dinâmica e organização de dados em formato tabular, buscando eficiência e clareza no código. As principais melhorias incluíram a busca por trechos de texto (título ou autor), o uso de matrizes de *strings* para exibir relatórios e a alocação dinâmica para gerenciar o acervo sem limite fixo.

2. Metodologia

O projeto foi desenvolvido no **Visual Studio Code (VSCode)**, executado no **Windows 11** e compilado com o **MinGW (GCC)**. A escolha dessas ferramentas foi motivada pela minha troca de sistema operacional, que na unidade passada era baseado em **Linux**.

A abordagem de desenvolvimento adotada foi incremental, priorizando a implementação e o teste de cada funcionalidade de forma isolada antes de integrar ao sistema principal. Inicialmente, foram criadas as funções básicas de cadastro e listagem, que serviram de base para a adição de recursos mais avançados, como a alocação dinâmica com redimensionamento automático do vetor de livros e a busca textual utilizando funções da biblioteca `<string.h>`. Essa estratégia garantiu um processo de desenvolvimento mais controlado e reduziu a ocorrência de erros complexos.

O código foi estruturado de forma modular, separando cada funcionalidade em funções específicas — como `cadastrarLivro()`, `listarLivros()`, `alugarLivro()` e `buscarLivro()` — além de rotinas auxiliares para limpeza de buffer, validação de entrada e redimensionamento de memória. Essa organização facilitou a manutenção e a leitura do código, permitindo que partes do programa fossem atualizadas sem comprometer outras. A modularização também contribuiu para a reutilização de trechos de código e para a padronização das operações, fortalecendo as boas práticas de programação em C.

3. Análise do Código

O Sistema de Gerenciamento de Biblioteca desenvolvido aplica de forma prática os principais conceitos da Unidade 2, com destaque para o uso de strings, estruturas de repetição aninhadas, matrizes, ponteiros e alocação dinâmica de memória. A seguir, descrevo como cada um desses recursos foi utilizado no funcionamento do programa.

Strings

As strings são fundamentais no sistema, pois quase todas as interações envolvem textos, como títulos, autores e nomes de usuários. Foram declaradas como vetores de caracteres (`char nome[100]`) e manipuladas com funções da biblioteca `<string.h>`, como `strcpy()` e `strncpy()` para cópia e truncamento, `strlen()` para validação, `strcmp()` para verificar se um livro está alugado, `strcspn()` para remover o `\n` de `fgets()` e `strstr()` para buscas parciais. A função `tolower()` da `<ctype.h>` foi usada para tornar as buscas insensíveis a maiúsculas e minúsculas.

Estruturas de Repetição Aninhadas

Essas estruturas aparecem principalmente na função `listarLivros()`, onde dois laços `for` percorrem livros e colunas da matriz de exibição. Também são usadas em outras partes do código, como na busca e conversão de textos, permitindo percorrer diferentes níveis de dados e garantindo a organização das informações.

Matrizes

As matrizes foram aplicadas dinamicamente na função `listarLivros()` por meio de uma estrutura tridimensional (`char ***lista`) que organiza os livros em linhas e campos em colunas. Cada célula é alocada dinamicamente e recebe uma string truncada para manter o formato tabular. Os laços duplos percorrem os dados e, ao final, a memória é liberada, assegurando eficiência e boa apresentação no terminal.

Ponteiros

Os ponteiros são amplamente utilizados, com destaque para `struct livro *biblioteca`, que armazena o endereço base do vetor dinâmico de livros. Graças a ele, o sistema ajusta seu tamanho conforme novos cadastros, sem limites fixos. Também há uso de ponteiros duplos e triplos em matrizes, permitindo manipular diretamente os dados em memória e aumentando a eficiência ao evitar cópias desnecessárias.

Alocação Dinâmica

A alocação dinâmica tornou o sistema flexível e escalável. A função `redimensionarBiblioteca()` usa `realloc()` para expandir o vetor quando o limite é atingido, dobrando sua capacidade e reduzindo o número de realocações. Há tratamento de erros caso `realloc()` falhe, e toda memória alocada, inclusive as matrizes da listagem, é liberada com `free()`, evitando vazamentos e garantindo desempenho consistente.

4. Dificuldades e Soluções

O maior desafio do projeto foi compreender e aplicar corretamente o uso de ponteiros e a memória dinâmica. Nos primeiros testes, o sistema apresentou falhas de segmentação e perda de dados durante as realocações, o que exigiu uma análise detalhada do comportamento dos endereços de memória. A solução veio com o estudo e aplicação adequada das funções `malloc()` e `realloc()`, além da validação rigorosa dos ponteiros e da liberação correta dos blocos alocados com `free()`. Essa etapa foi essencial para garantir estabilidade ao programa e evitar erros críticos de execução.

Outro ponto de dificuldade foi a formatação das strings e da tabela de listagem, que inicialmente apresentava desalinhamentos e inconsistências de espaçamento. Para resolver, foram criadas funções auxiliares específicas para truncar e ajustar o tamanho dos textos, além do uso de laços aninhados para organizar os dados em formato tabular.

Essas soluções aprimoraram tanto a aparência quanto o desempenho do sistema, consolidando o aprendizado sobre controle de memória, manipulação de dados textuais e reforçando a importância da modularização e dos testes constantes durante o desenvolvimento.

5. Conclusão

O desenvolvimento do projeto da Unidade 2 representou um avanço marcante na consolidação dos conhecimentos adquiridos em linguagem C. Foi possível aplicar de forma prática conceitos essenciais como manipulação de strings, uso de ponteiros, matrizes e alocação dinâmica de memória, resultando em um sistema funcional, flexível e escalável.

Durante o processo, ficou evidente a importância de planejar e modularizar o código, além de adotar uma abordagem que privilegia o tratamento de erros e a liberação correta da memória. O aprendizado obtido com o tratamento de falhas e a depuração de ponteiros incorretos foi essencial para aprimorar a lógica e a estrutura do programa.

Como melhorias futuras, pretendo implementar funcionalidades de remoção de livros e armazenamento em arquivos, o que permitirá manter os registros mesmo após o encerramento da execução. Além disso, planejo desenvolver uma interface mais interativa, com menus organizados e mensagens mais intuitivas para o usuário.

6. Perguntas Orientadoras

1. Quais conceitos da U2 foram aplicados?

Manipulação de *strings*, matrizes, repetições aninhadas, ponteiros e alocação dinâmica, aplicados para criar um sistema flexível e escalável.

2. Como a organização do código facilita a manutenção?

A separação em funções e a padronização de nomes e comentários tornam o código legível e fácil de expandir.

3. Quais foram os principais desafios técnicos?

Compreender a lógica de ponteiros e evitar vazamentos de memória. A solução veio com verificações e testes progressivos.

4. Como foram implementadas as matrizes?

Com laços duplos e alocação dinâmica, armazenando campos textuais em estruturas tridimensionais para gerar tabelas.

5. Qual a estratégia de gerenciamento de memória?

Alocação sob demanda com `malloc()` e `realloc()`, sempre acompanhadas de verificações e liberações com `free()`.

6. Como garantir a ausência de vazamentos?

Cada alocação possui desalocação correspondente e foi validada por testes repetidos.

7. Quais vantagens da alocação dinâmica?

Remoção de limites fixos e adaptação do tamanho conforme o uso, otimizando recursos.

8. Como os ponteiros aumentaram a eficiência?

Permitiram manipular dados diretamente na memória e passar estruturas por referência, reduzindo cópias e acelerando execuções.